Technical Assessment: Pipeline Builder Implementation
Candidate: Karan Malviya
Role: Frontend Engineering

Phase 1: Node Abstraction & Styling

Objective: Create a maintainable architecture to eliminate redundant code.

- **Base Node Architecture:** I developed a reusable BaseNode component in /frontend/src/nodes. This abstraction manages shared logic for headers, handle positioning, and state, reducing boilerplate for new node creation.
- **Custom Node Suite:** Leveraging the new abstraction, I implemented five additional nodes to demonstrate system flexibility: **Note, Timer, Condition, Filter, and Transform**.
- **Unified Design System:** I applied a cohesive visual style across all components, featuring a professional purple-themed interface and consistent layout patterns.

Phase 2: Advanced Text Node Logic

Objective: Enhance user experience through dynamic UI updates and variable mapping.

- **Dynamic Auto-Resizing:** The Text node is programmed to automatically adjust its width and height as the user inputs text, ensuring constant visibility of the content.
- **Variable Extraction & Dynamic Handles:**
    - The system monitors the text input for valid JavaScript variables wrapped in double curly brackets, such as {{ input }}.
    - Upon detection, a new **Handle** is instantly rendered on the left side of the node to facilitate dynamic data connections.

Phase 3: Backend Integration & Graph Validation

Objective: Connect the frontend pipeline to a FastAPI backend for structural analysis.

- **Frontend Submission:** I updated submit.js to capture the current state of all nodes and edges. This payload is sent to the /pipelines/parse endpoint when the "Submit Pipeline" button is clicked.
- **Backend Logic:** I modified the FastAPI /pipelines/parse endpoint in main.py to perform the following:
    - Calculate the total number of nodes and edges.
    - Determine if the provided structure forms a **Directed Acyclic Graph (DAG)**.

- **User Feedback:** The frontend receives the response in the format {num_nodes: int, num_edges: int, is_dag: bool} and triggers a user-friendly alert displaying the results.

How to Run

1. **Frontend:** Navigate to /frontend and run npm i then npm start.
2. **Backend:** Navigate to /backend and run uvicorn main:app --reload.